Sets, Constants and Variables

Klaus Draeger

Lecture 2: Sets, Constants and Variables

The aim of this lecture is to:

- Give more detail of the **data** in abstract machines:
  - New types defined in the SETS section,
  - Derived sets defined in the CONSTANTS and PROPERTIES sections,
  - Variabled defined in the VARIABLES, INVARIANT, INITIALISATION sections.
- Connect them to the system's **states**:
  - Variables defining the current state
  - Invariants defining the valid states
  - Initialisation defining the starting state

Use of sets in abstract machines

- The initial stage of the B-method is to formalise the system requirements
- This is done by creating an abstract machine describing
  - The **state variables** and their **types**
  - The **invariants** defining the **valid** states
  - The **operations** to transform and update the state.
- This is a **formalisation** of the requirements
  - Ignores as much implementation detail as possible
  - Using sets since they work very well for this

Use of sets in abstract machines

Our machines can contain 3 kinds of base sets:

- Built-in sets: INTEGER, NAT, NAT1
  - We already know these well enough.
- **Enumerated** sets
  - Finite collections of symbolic values
- **Deferred** sets
  - only partially defined
  - details to be added later

Enumerated Sets

$ENUMERATED_SET = \{element_1, element_2, \ldots, element_n\}$

- Entirely new "symbolic" elements represented by names
- Can be implemented as enums/ints/strings/subclasses/...

Some examples:

- Possible question answers.
  ANSWER = { yes, no }
- Possible cursor key presses and movements.
  CURSOR_KEY = { Up_Key, Down_Key, Left_Key, Right_Key }
- Messages output by a self service till.

$$MESSAGE = \{\ Success,\ Are_You_Using_Your_Own_Bag,\ Have_You_Swiped_Your_Nectar_Card,\ Error_Unknown_Item_In_Bagging_Area\ \}$$

Deferred Sets

- When a **"deferred"** set is declared the actual elements of the set are **not specified**
- So we can abstract away from details of:
  - What the actual elements of the set are
  - How they are represented.
- These details have to eventually be added
- Will often be **partially** defined first by
  - Defining some **constants** to be included in the set
  - Still leaving open what else it contains

Deferred Sets Example

```
MACHINE Organisation
    SETS
        // We are leaving the set of members open for now...
        Members
    CONSTANTS
        // But they include these:
        TechLead, Treasurer, Publicist
    PROPERTIES
        TechLead : Members & Treasurer : Members &
        Publicist : Members
END
```

- Unlike members of a SET these constant values are not necessarily different
- E.g. the Treasurer and Publicist could be the same person
- If this is not wanted we need to explicitly forbid it

Deferred Sets Example

```
MACHINE Organisation
    SETS
        // We are leaving the set of members open for now...
        Members
    CONSTANTS
        // But they include these:
        TechLead, Treasurer, Publicist
    PROPERTIES
        TechLead : Members & Treasurer : Members &
        Publicist : Members & TechLead /= Treasurer &
        TechLead /= Publicist & Treasurer /= Publicist
END
```

Derived Sets: Constants

- The type sets in the SETS section provide the basis for data definitions
- We will generally need more complex structures
- For this we define derived sets in the CONSTANTS and PROPERTIES sections.
- This uses the various set operations
  - Set union, intersection, difference
  - Cartesian product
  - Power set
- Plus more that we will see later

Derived Sets Example

```
MACHINE ChessGame
    CONSTANTS
        ChessBoard, WhiteKingStart, BlackPawnStart // (etc)
    PROPERTIES
        // Positions on the board: pairs (x,y), both in 1..8
        Chessboard = 1..8 * 1..8 &
        // The white king starts on E1, i.e. x=5, y=1
        WhiteKingStart : ChessBoard &
        WhiteKingStart = (5,1) &
        // The black pawns start in row 7, i.e. x in 1..8 and y=7
        BlackPawnStart <: ChessBoard &
        BlackPawnStart = 1..8 * {7}
END
```

Derived Sets Example

```
MACHINE FamilyGatherings
    SETS
        Family = {Alex, Beth, Carl, Denise, Emily}
    CONSTANTS
        Conflict, PossibleGatherings
    PROPERTIES
        // Two family members don't get along
        Conflict <: Family & Conflict = {Beth, Carl} &
        // Can meet with any family members,
        // just not those 2 together
        PossibleMeetings <: POW(Family) &
        PossibleMeetings = {S | S <: Family & Conflict /<: S}
END
```

Adding Variables

Finally we need to define the machine's **variables**

```
MACHINE ColourText
    SETS
        Colours = {Black, White, Red, Green, Blue}
    CONSTANTS
        BW, Primary
    PROPERTIES
        BW <: Colours & BW = {Black, White} &
        Primary <: Colours & Primary = {Red, Green, Blue} &
    VARIABLES
        foreground, background
    INVARIANT
        foreground : Colours & background : Colours
    INITIALISATION
        foreground := black || background := white
END
```

One Note on Notation

Note that we use different separators in the different parts of the machine:

- **Complete declarations** (SETS, OPERATIONS) are separated by **semicolons** (;)
- CONSTANTS and VARIABLES are simply lists of items separated by **commas** (,)
- PROPERTIES and INVARIANT consist of statements separated by **logical and** (&)
- INITIALISATION consists of assignments separated by **parallel composition** (||)

Categories of System States

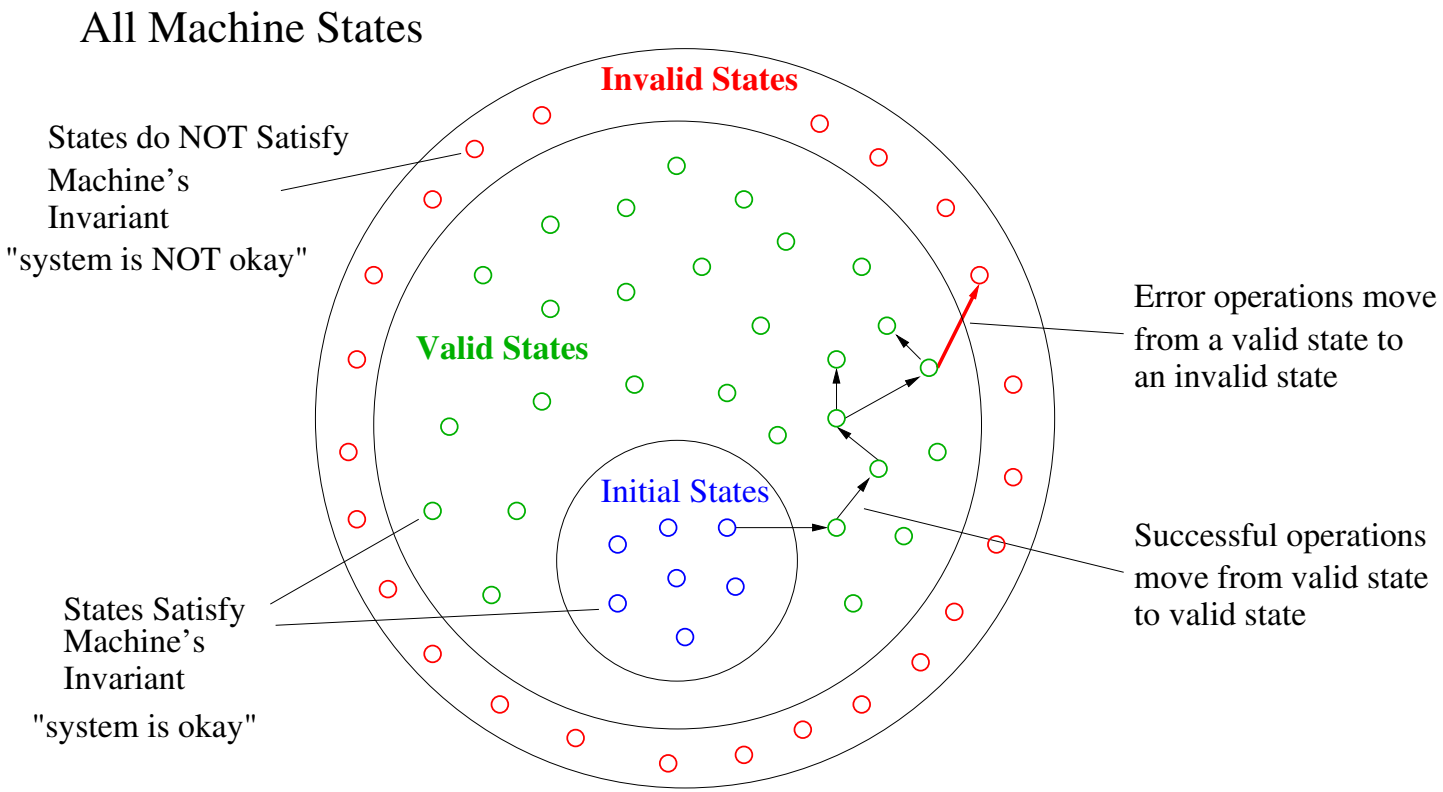


Figure: All Possible System States

Categories of System States

- Any system can be in a variety of **states**
  - Defined by the values of its variables
  - For example, an airplane's state could include
    - Its current altitude
    - Its cabin pressure
    - The state of each door (open/closed)
- Each state is either **valid** or **invalid**
  - Validity is defined by a condition (the invariant)
  - The airplane's invariant might include:
    - **If** the current altitude is >0 **then** all doors should be closed
  - Our goal is to ensure that all operations preserve validity
- An **initial** state is one in which the system can start
  - These should all be valid!
  - Our systems will have a **single** initial state

Example: Stack States

Consider a **stack** with a maximum size, then the:

- **System invariant** is that
  "zero $\leq$ *the number of items* $\leq$ *maximum size"*.
- **Valid states** are those that satisfy the invariant
- **Initial state** would be the **empty** stack.
  - This also satisfies the invariant.
- **Invalid states** are those that do not satisfy the invariant:
  number of items $<$ zero, or
  number of items $>$ maximum size.

Executing System Operations

An **execution** of the system:

- Starts from one of the **initial states**.
  - This needs to be a **valid** state.
- When an **operation** is performed, the system moves to another state.
  - This should **preserve validity**.
- As long as those conditions are satisfied (initial states are valid, and operations preserve validity) the state will always be valid.

Example: Executing Stack Operations

For the stack with a maximum size:

- Execution starts from the **empty** stack
  (which is a valid state).
- A **push** operation can be performed whenever the stack is not **full**
  (if it was full pushing would lead to an invalid state)
- A **pop** operation can be performed whenever the stack is not **empty**
  (if it was empty popping would lead to an invalid state)

Conditions like above for push and pop need to be part of the operations' **precondition**

Specifying an Operation

The questions to consider when **specifying an operation** are:

Pre-Condition components:

- What are its **data input parameters**?
- What **preconditions** must they satisfy?
- What **preconditions** must the **state** satisfy?

Post-Condition components:

- What part of the **state is altered, and how**?
- What part of the **state remains the same**?
- What are the **outputs** (data and/or reports)?

Operation Details

In general, specifications of operations require the following information:

- "**Operation Interface**" – is defined by its
  - **Name**.
  - Optional list of **Input parameters** in parentheses.
    If the operation has none, just omit the list (including parentheses).
  - Optional list of **Output variables**.
    The results of the operation are assigned to these.
- **Precondition** constraining the input parameters and machine state.
  Must be true for the operation to be executed.
- **Body** of the operation – describes its effect.
  - assigns new values to some **state variables**
  - assigns results to the **output variables**.

Abstract Machine Notation (AMN)

The **Abstract Machine Notation** (AMN) consists of several elements

- Predicate logic:
  used to express properties of data (e.g. invariants, preconditions)
- Expressions:
  formulas describing data (its type and value)
- Substitutions:
  describe the **dynamic** aspect, i.e.state changes.

Abstract Machine Notation (AMN)

AMN includes the following **programming language constructs**:

- **assignment** including **multiple assignment**:
  xx,yy,zz := yy,zz,0
  (equivalent to xx:=yy || yy:=zz || zz:=0)
- **conditional** constructs (IF-THEN-ELSE-END)
- the "**do nothing**" command: SKIP,
- **block structure**: BEGIN . . . END, like blocks {. . . } in Java
- **parallel execution**: ||'

Conditionals

General IF statements:
IF C1 THEN S1 ELSIF C2 THEN S2 ELSIF ... ELSE S END

- Checks the conditions in order
- Picks the first which is true
- Executes the corresponding branch
- If none is true, executes the final (ELSE) branch (if any)

Tangent: Beyond Valid States

- We have defined correctness in terms of **valid states**
- This is an example of a **safety property**:
  The system is **always** in a "good state"
- There are other kinds of property that can matter:
  - **Liveness**: The system will **eventually** reach a "goal" state
  - **Responsiveness**: **Whenever** some event happens, the system (eventually) reacts
- Properties like these can be defined using **temporal logics**